

INERTIA.JS MASTERY NOTES

STACK: LARAVEL + INERTIA 2 + SVELTE 5 + VITE

Based on the Laracasts "Build Modern Laravel Apps Using Inertia.js" course outline, adapted from Vue 3 to Svelte 5 runes mode.

HEADS UP — The original Laracasts course uses Vue 3 and Laravel Mix. These notes translate every code example to your stack: Svelte 5 (runes), Inertia 2, and Vite. Any time you see something in the course that says "Vue", you should mentally swap it for the Svelte equivalent shown here.

1. WHAT INERTIA.JS ACTUALLY IS

THE MODERN MONOLITH

Inertia is not a framework. It is not a Vue/Svelte replacement. It is **glue** between a server-side framework (Laravel) and a client-side UI framework (Svelte, in our case). It lets you build a single-page application without building a separate API.

The core insight: most apps build an API + a separate SPA because that is "the modern way", but for monolithic apps, that split is overkill. You end up writing two apps that talk to each other, dealing with OAuth, CORS, separate repos, and duplicated validation logic. Inertia removes that split entirely.

WHAT STAYS THE SAME

- Routes still live in `routes/web.php`
- Controllers, models, middleware, validation, auth, sessions — all standard Laravel
- CSRF protection works automatically
- You never write a JSON API by hand

WHAT CHANGES

- Instead of returning a Blade view, you return an `Inertia::render()` response
- Instead of `<a>` tags, you use `<Link>` components that intercept clicks
- Instead of full page reloads, the next page comes back as JSON and swaps in client-side

HOW THE MAGIC WORKS

When the browser first loads, you get a normal HTML response containing your Svelte app bundle. On every subsequent navigation, Inertia intercepts the click, sends an AJAX request to the same Laravel route, and Laravel returns **JSON** instead of HTML because Inertia sets an `X-Inertia` header. The JSON payload contains the page component name and the props. Inertia swaps the current page component for the new one. No full reload.

THE MENTAL MODEL

Think of it like Blade views, except:

- Your "view" is a `.svelte` file in `resources/js/Pages/`
- Your controller passes data the same way, but via `Inertia::render()` instead of `view()`
- Navigation between pages feels instant

2. PAGES

RETURNING A PAGE FROM A ROUTE

Two equivalent ways — both work, `Inertia::render()` is the canonical one:

```
<?php
use Inertia\Inertia;

Route::get('/', function () {
    return Inertia::render('Home', [
        'name' => 'Benjamin',
        'tweets' => ['First tweet', 'Second tweet'],
    ]);
});

// Or the helper function (same thing)
Route::get('/', fn() => inertia('Home', ['name' => 'Benjamin']));
```

PAGE COMPONENT CONVENTIONS

- Pages live in `resources/js/Pages/`
- Names are PascalCase (capitalized) — e.g., `Home.svelte`, `Users/Index.svelte`
- Subdirectories work: `Inertia::render('Users/Index')` resolves to `resources/js/Pages/Users/Index.svelte`

RECEIVING PROPS IN SVELTE 5

In Svelte 5 (runes mode), you receive props via the `$props()` rune:

```
<script>
  let { name, tweets } = $props();
</script>

<h1>Hello, {name}!</h1>

<ul>
  {#each tweets as tweet}
    <li>{tweet}</li>
  {/each}
</ul>
```

SVELTE 5 VS OLDER SVELTE — In Svelte 3/4 you would write "export let name". In Svelte 5 with runes, you destructure from `$props()`. The course uses Vue's `defineProps` — same idea, different syntax.

THE INERTIA PAGE MENTAL MODEL

Inertia pages are the Svelte equivalent of Blade views. Controller passes data → page receives data → page renders. The difference is the page is a real, reactive component, not a templated HTML string.

3. INERTIA LINKS

WHY YOU CANNOT USE PLAIN ANCHOR TAGS

A normal `<a href="/about">` triggers a full page refresh. The browser throws away your Svelte app, downloads a new HTML document, and rebuilds everything from scratch. That breaks the SPA feel entirely — flashing screens, lost state, lost scroll position.

THE LINK COMPONENT

```
<script>
  import { Link } from '@inertiajs/svelte';
</script>

<nav>
  <Link href="/">Home</Link>
  <Link href="/users">Users</Link>
  <Link href="/settings">Settings</Link>
</nav>
```

When a Link is clicked, Inertia:

- Intercepts the click event
- Sends an AJAX request to the URL with an `X-Inertia: true` header
- Receives JSON back containing `{ component, props, url, version }`
- Updates the URL with `history.pushState` (so back button works)
- Swaps the current page component for the new one
- No full reload, no lost state outside the page itself

THE HANDLEINERTIAREQUESTS MIDDLEWARE

Laravel detects the `X-Inertia` header and the `HandleInertiaRequests` middleware (already registered by the starter kit in `bootstrap/app.php`) returns JSON instead of HTML for that request.

EXTRACTING A SHARED NAV COMPONENT

Put reusable components in `resources/js/Shared/`. Convention only — not enforced by Inertia.

```
<!-- resources/js/Shared/Nav.svelte -->
<script>
  import { Link } from '@inertiajs/svelte';
</script>

<nav class="flex space-x-4 bg-gray-800 p-4 text-white">
  <Link href="/" class="hover:text-blue-300">Home</Link>
  <Link href="/users" class="hover:text-blue-300">Users</Link>
  <Link href="/settings" class="hover:text-blue-300">Settings</Link>
</nav>

<!-- resources/js/Pages/Home.svelte -->
<script>
  import Nav from '../Shared/Nav.svelte';
  let { name } = $props();
</script>
```

```
<Nav />  
<h1>Hello, {name}!</h1>
```

4. PROGRESS INDICATORS

BUILT INTO INERTIA 2 — NO SEPARATE PACKAGE

The old course uses `@inertiajs/progress`. That package is deprecated. Inertia 2 ships with the progress bar baked in.

CONFIGURING THE PROGRESS BAR

In `resources/js/app.ts`, the progress option is passed to `createInertiaApp()`:

```
import { createInertiaApp } from '@inertiajs/svelte';
import { mount } from 'svelte';

createInertiaApp({
  resolve: name => {
    const pages = import.meta.glob('./Pages/**/*.svelte', { eager: true });
    return pages[`./Pages/${name}.svelte`];
  },
  setup({ el, App, props }) {
    mount(App, { target: el, props });
  },
  progress: {
    color: '#29d', // bar color
    delay: 250, // ms before bar appears (avoids flicker on fast requests)
    includeCSS: true, // inject default CSS
    showSpinner: false, // spinner in top-right corner
  },
});
```

WHY THE DELAY — Fast requests under 250ms don't show the bar. Avoids ugly flicker on local-fast network responses while still showing the bar on slow ones.

5. PERFORMING NON-GET REQUESTS WITH LINK

WHY LOGOUT MUST BE POST

GET requests should be safe and idempotent. A logout URL accessible via GET means anyone can log you out by tricking you into clicking a link, or pre-fetchers and crawlers can accidentally log you out. POST + CSRF protection prevents both.

METHOD AND DATA PROPS

```
<script>
  import { Link } from '@inertiajs/svelte';
</script>

<Link href="/logout" method="post" as="button">
  Log out
</Link>

<Link
  href="/tweets/{tweet.id}/like"
  method="post"
  data={{ user_id: 42 }}
  as="button"
>
  Like
</Link>
```

THE "AS" PROP

By default `<Link>` renders as an `<a>` tag, even for POST. That breaks two things: cmd-click "Open in new tab" tries to GET the URL (and fails), and right-click "Copy link address" exposes a URL that does nothing useful. Setting `as="button"` renders a `<button>` instead, which behaves correctly.

CSRF TOKENS ARE AUTOMATIC

Inertia uses Axios under the hood. Axios reads the `XSRF-TOKEN` cookie that Laravel sets automatically and sends it as the `X-XSRF-TOKEN` header. The `VerifyCsrfToken` middleware sees that and is happy. You never write `@csrf` in an Inertia form.

STYLING A BUTTON TO LOOK LIKE A LINK

Browsers add default button styles. Reset them with Tailwind:

```
<Link
  href="/logout"
  method="post"
  as="button"
  class="text-blue-500 hover:underline bg-transparent border-0 p-0 cursor-pointer"
>
  Log out
</Link>
```

6. PRESERVE SCROLL POSITION

THE PROBLEM

By default, every Inertia navigation resets scroll to the top. That makes sense for *different* pages, but breaks the experience when you're refreshing the same page — e.g., clicking a like button, changing a filter, or paginating partway down a long list.

THE FIX

```
<Link href="/tweets" preserveScroll>Refresh</Link>

<!-- or on a programmatic request -->
<script>
  import { router } from '@inertiajs/svelte';

  function refresh()
    router.get('/tweets', {}, { preserveScroll: true });

</script>
```

WHEN TO USE IT — Anytime the user's position on the page would be lost for no reason. Filter inputs, like buttons, "load more" — all candidates for `preserveScroll`.

7. ACTIVE LINKS

THE \$PAGE STORE

Inertia exposes a Svelte store called `page`. Subscribe to it with `$page` to get the current page state — URL, component name, props, version.

```
<script>
  import { page } from '@inertiajs/svelte';
  import { Link } from '@inertiajs/svelte';
</script>

<Link
  href="/users"
  class={$page.url.startsWith('/users') ? 'font-bold underline' : ''}
>
  Users
</Link>
```

URL VS COMPONENT MATCHING

Matching by URL breaks when query strings change — `/users?search=ben` would not match `startsWith("/users")` depending on how you wrote it. Component matching is more robust:

```
<Link
  href="/users"
  class={$page.component === 'Users/Index' ? 'font-bold' : ''}
>
  Users
</Link>
```

EXTRACTING A REUSABLE NAVLINK COMPONENT

Repeating that conditional class for every link is gross. Wrap it.

```
<!-- resources/js/Shared/NavLink.svelte -->
<script>
  import { page } from '@inertiajs/svelte';
  import { Link } from '@inertiajs/svelte';

  let { href, active, children, ...rest } = $props();

  // active prop is either a component name string or a URL prefix
  let isActive = $derived(
    active
      ? $page.component === active || $page.url.startsWith(active)
      : $page.url === href
  );
</script>

<Link
  {href}
  class={isActive ? 'font-bold underline' : 'hover:underline'}
  {...rest}
>
  {@render children()}
</Link>
```

Usage:

```
<NavLink href="/" active="Home">Home</NavLink>  
<NavLink href="/users" active="Users/Index">Users</NavLink>
```

SVELTE 5 NOTE — In older Svelte you would use <slot>. In Svelte 5, slots are replaced by snippets. children is a snippet automatically available as a prop, rendered with {@render children()}.

8. LAYOUT FILES

THE SCALING PROBLEM

Importing `<Nav />` in every single page works for 3 pages. For 30 pages, it's painful. For 300, it's untenable. A Layout component wraps every page in shared chrome.

CREATING A LAYOUT

```
<!-- resources/js/Shared/Layout.svelte -->
<script>
  import Nav from './Nav.svelte';
  let { children } = $props();
</script>

<div class="min-h-screen bg-gray-100">
  <Nav />
  <main class="container mx-auto p-6">
    {@render children()}
  </main>
</div>
```

WRAPPING EACH PAGE

```
<!-- resources/js/Pages/Home.svelte -->
<script>
  import Layout from '../Shared/Layout.svelte';
  let { name } = $props();
</script>

<Layout>
  <h1>Hello, {name}!</h1>
</Layout>
```

LIMITATION — Notice the page imports the layout itself. This works, but the layout is destroyed and rebuilt on every navigation because it lives inside the page component. Anything stateful in the layout (audio player, sidebar scroll, open dropdown) gets wiped. We fix this in section 11: Persistent Layouts.

9. SHARED DATA

THE PROP-DRILLING PROBLEM

The logged-in user's name should appear in your Nav. But the Nav is rendered by the Layout, which is included by every page. Passing the user via props from every single controller is insane.

THE SHARE() METHOD

The `HandleInertiaRequests` middleware has a `share()` method that adds data to every Inertia response automatically.

```
<?php
// app/Http/Middleware/HandleInertiaRequests.php

namespace App\Http\Middleware;

use Illuminate\Http\Request;
use Inertia\Middleware;

class HandleInertiaRequests extends Middleware
{
    public function share(Request $request): array
    {
        return array_merge(parent::share($request), [
            'auth' => [
                'user' => fn() => $request->user()
                    ? [
                        'id' => $request->user()->id,
                        'name' => $request->user()->name,
                        'username' => $request->user()->username,
                    ]
                    : null,
            ],
            'flash' => [
                'success' => fn() => $request->session()->get('success'),
                'error' => fn() => $request->session()->get('error'),
            ],
        ]);
    }
}
```

ERRORS ARE SHARED AUTOMATICALLY

The base `Middleware` class already shares validation errors. That's why you can access `$page.props.errors` from any page without doing anything extra.

ACCESSING SHARED DATA IN SVELTE

```
<script>
import { page } from '@inertiajs/svelte';

let user = $derived($page.props.auth.user);
</script>
```

```
{#if user}
  <span>Hello, {user.name}</span>
{:else}
  <Link href="/login">Log in</Link>
{/if}
```

SECURITY WARNING — NEVER share a whole user model like `fn() => $request->user()`. That dumps the password hash, email, remember_token, stripe_customer_id — every single column — into a JSON blob visible in DevTools. Always whitelist the fields you actually need. This is also covered in section 15.

NAMESPACING

Notice the structure: `auth.user.name` instead of just `user`. Namespaces prevent shared-data props from colliding with page-specific props. You will be glad you did this when *page-level* `user` shows up later for editing.

10. GLOBAL COMPONENTS — A SVELTE NON-ISSUE

WHY VUE NEEDS THIS

In the Vue course, Jeffrey shows how to globally register `<Link>` and `<Head>` so you don't have to import them on every single page. Vue has a `app.component()` API for exactly this.

SVELTE DOES NOT HAVE THIS

Svelte has no global component registry. You import every component where you use it. That sounds worse, but it actually buys you:

- Better tree-shaking by default — unused components are not bundled
- IDE goto-definition just works — every usage points to a clear import
- No "magic" components appearing from nowhere
- No naming conflicts with HTML elements

WHAT THIS MEANS FOR YOU

Just keep doing this in every page:

```
<script>
  import { Link, Head } from '@inertias/svelte';
  import Layout from '../Shared/Layout.svelte';
</script>
```

It feels repetitive, but it's the Svelte way and Cursor's autocomplete makes it a non-issue — start typing `Link` and it auto-imports.

11. PERSISTENT LAYOUTS

THE REAL PROBLEM

With template wrapping (`<Layout>...content...</Layout>`), the Layout is a child of the page. When you navigate from Home to Users, the entire Home page is unmounted — and its child Layout goes with it. The new Users page mounts its own Layout. Result: layout state resets every navigation.

THE FIX — LAYOUT AS PAGE METADATA

In Inertia Svelte 2, you set the layout as a top-level export from a module-scoped script. The framework reads this, wraps the page in the layout, and **keeps the layout instance alive** across navigations.

```
<!-- resources/js/Pages/Users/Index.svelte -->
<script module>
  import Layout from '../Shared/Layout.svelte';
  export const layout = Layout;
</script>

<script>
  let { users } = $props();
</script>

<h1>Users</h1>
{#each users as user}
  <p>{user.name}</p>
{/each}
```

WHAT CHANGED

- No more `<Layout>...</Layout>` wrap in the template
- The page is now a child of the Layout (handled by Inertia), not the other way around
- Layout state (audio player, sidebar scroll, modals) persists between navigations

SVELTE 5 SYNTAX DETAIL — Use `<script module>` (Svelte 5) — older Svelte uses `<script context="module">`. Both work in transitional codebases, but Svelte 5 prefers the new spelling.

12. DEFAULT LAYOUTS

SETTING A DEFAULT FOR EVERY PAGE

Exporting `Layout` from every page is still repetitive. Set it once in `app.ts`, and pages opt out only when they need to.

```
// resources/js/app.ts
import { createInertiaApp } from '@inertiajs/svelte';
import { mount } from 'svelte';
import Layout from './Shared/Layout.svelte';

createInertiaApp({
  resolve: name => {
    const pages = import.meta.glob('./Pages/**/*.svelte', { eager: true });
    const page = pages[`./Pages/${name}.svelte`];

    // Assign default layout if the page didn't define one
    if (page.default.layout === undefined) {
      page.default.layout = Layout;
    }

    return page;
  },
  setup({ el, App, props }) {
    mount(App, { target: el, props });
  },
});
```

OPTING OUT OF THE LAYOUT

For pages like Login that should not have the Nav (no logged-in user yet), explicitly set `layout = null`:

```
<!-- resources/js/Pages/Auth/Login.svelte -->
<script module>
  export const layout = null;
</script>

<script>
  /* login form here */
</script>
```

WHY === UNDEFINED — Use strict equality with undefined, not a falsy check. Otherwise `layout = null` gets overwritten by your default, defeating the opt-out.

13. CODE SPLITTING AND DYNAMIC IMPORTS

VITE REPLACES LARAVEL MIX

The course teaches Mix's `.extract()` for splitting vendor bundles. That's obsolete. Vite handles bundle splitting automatically and far more intelligently. You just choose between *eager* (one big bundle) and *lazy* (chunk per page) loading.

EAGER — ALL PAGES IN ONE BUNDLE

```
const pages = import.meta.glob('./Pages/**/*.svelte', { eager: true });  
  
return pages[`./Pages/${name}.svelte`];
```

Synchronous. Every page is bundled into the main JS file. Faster navigations after initial load, but a bigger initial download.

LAZY — CHUNK PER PAGE

```
const pages = import.meta.glob('./Pages/**/*.svelte');  
  
return pages[`./Pages/${name}.svelte`]();  
// note the () ^^ — each glob entry is now a function returning a Promise
```

Returns a Promise. Each page becomes its own chunk that's only downloaded when that page is first visited.

WHICH ONE TO USE

- Small app (under 20 pages): use `eager: true`. Simpler, faster, no waterfall on first navigation.
- Large app (50+ pages): drop `eager` and let Vite split chunks. Smaller initial download.
- For your Twitter clone: start eager. Switch later if the initial bundle gets fat.

14. DYNAMIC TITLE AND META TAGS

THE HEAD COMPONENT

Inertia exposes a `<Head>` component that injects elements into the document's `<head>`. Use it for per-page titles and meta tags.

```
<script>
  import { Head } from '@inertiajs/svelte';
  let { user } = $props();
</script>

<Head title="{user.name}'s Profile" />

<h1>{user.name}</h1>
```

MULTIPLE META TAGS

```
<Head>
  <title>{user.name}'s Profile</title>
  <meta name="description" content="Profile page for {user.name}" />
  <meta property="og:title" content="{user.name} on Twitter Clone" />
  <meta property="og:image" content="{user.avatar_url}" />
</Head>
```

A DEFAULT TITLE TEMPLATE

Set a template once in `app.ts` so every page automatically suffixes the app name:

```
createInertiaApp({
  title: title => `${title} - Twitter Clone`,
  // ... rest of config
});
```

Now `<Head title="Users" />` becomes `Users - Twitter Clone`. Pages with no title fall back to `"Twitter Clone"`.

15. A CRITICAL SPA SECURITY CONCERN

THE BIG REALIZATION

Every prop you pass to an Inertia page lands in the JSON payload, visible in DevTools → Network tab. So if you do this:

```
Route::get('/users', fn() => Inertia::render('Users/Index', [
    'users' => User::all(), // BAD
]));
```

You just leaked every column of every user to the client: password hashes, remember tokens, email addresses, `stripe_customer_id`, `two_factor_secret` — everything.

TWO DISTINCT PROBLEMS

- **Over-fetching** — sending more data than the view needs (slow + wasteful)
- **Exposing secrets** — sending data that should never reach the client

FIX OPTION 1 — EXPLICIT MAPPING

```
Route::get('/users', fn() => Inertia::render('Users/Index', [
    'users' => User::all()->map(fn($user) => [
        'id'      => $user->id,
        'name'     => $user->name,
        'username' => $user->username,
        'avatar'  => $user->avatar_url,
    ]),
]));
```

FIX OPTION 2 — API RESOURCES (CLEANER FOR REUSE)

```
<?php
// app/Http/Resources/UserResource.php

namespace App\Http\Resources;

use Illuminate\Http\Resources\Json\JsonResource;

class UserResource extends JsonResource
{
    public function toArray($request): array
    {
        return [
            'id'      => $this->id,
            'name'     => $this->name,
            'username' => $this->username,
            'avatar'  => $this->avatar_url,
        ];
    }
}
```

```
use App\Http\Resources\UserResource;

Route::get('/users', fn() => Inertia::render('Users/Index', [
    'users' => UserResource::collection(User::all()),
]));
```

```
1));
```

VERIFY IN DEVTOOLS

Open Network tab, click any Inertia request, check the JSON response. If you see `password`, `remember_token`, or anything you didn't explicitly include — stop and fix it before going further.

16. PAGINATION

REPLACING ALL() WITH PAGINATE()

```
Route::get('/users', fn() => Inertia::render('Users/Index', [
  'users' => User::paginate(10),
]));
```

The shape of `users` in your Svelte page changes from a flat array to:

```
{
  data: [ { id, name, ... }, ... ],
  current_page: 1,
  last_page: 5,
  per_page: 10,
  total: 47,
  links: [
    { url: null,          label: '&laquo; Previous', active: false },
    { url: '...?page=1',  label: '1',             active: true  },
    { url: '...?page=2',  label: '2',             active: false },
    { url: '...?page=2',  label: 'Next &raquo;',      active: false },
  ],
}
```

THE CRUCIAL MAP() TRAP

You want to whitelist columns AND paginate. The naive attempt destroys pagination:

```
// BAD - ->map() on a paginator returns a plain Collection.
// You lose data, links, current_page, etc.
'users' => User::paginate(10)->map(fn($u) => [
  'id'   => $u->id,
  'name' => $u->name,
]),
```

Use `->through()` instead — it maps over the current slice but **keeps the paginator wrapper intact**:

```
'users' => User::paginate(10)->through(fn($u) => [
  'id'   => $u->id,
  'name' => $u->name,
]),
```

ITERATING IN SVELTE

```
<script>
  import { Link } from '@inertiajs/svelte';
  let { users } = $props();
</script>

<table>
  <thead>
    <tr><th>Name</th></tr>
  </thead>
  <tbody>
    {#each users.data as user}
      <tr><td>{user.name}</td></tr>
    {/each}
  </tbody>
```

```

</table>

<nav class="flex space-x-2 mt-4">
  {#each users.links as link}
    {#if link.url}
      <Link
        href={link.url}
        class={link.active ? 'font-bold text-blue-700' : 'text-gray-600'}
      >
        {@html link.label}
      </Link>
    {:else}
      <span class="text-gray-300">{@html link.label}</span>
    {/if}
  {/each}
</nav>

```

WHY @HTML ON LINK.LABEL — Laravel returns « and » HTML entities in the label. Plain text rendering shows the literal entity. Use {@html} to render them as the « » characters.

EXTRACTING A REUSABLE PAGINATION COMPONENT

```

<!-- resources/js/Shared/Pagination.svelte -->
<script>
  import { Link } from '@inertiajs/svelte';
  let { links } = $props();
</script>

<nav class="flex space-x-2 mt-4">
  {#each links as link}
    {#if link.url}
      <Link
        href={link.url}
        class={link.active ? 'font-bold text-blue-700' : 'text-gray-600'}
        preserveScroll
      >
        {@html link.label}
      </Link>
    {:else}
      <span class="text-gray-300">{@html link.label}</span>
    {/if}
  {/each}
</nav>

<!-- in a page -->
<Pagination links={users.links} />

```

17. FILTERING, STATE, AND QUERY STRINGS

THE GOAL

A search box that filters a paginated list. Typing in the box hits the server, returns filtered results, updates the URL with `?search=ben`, and preserves the input value across pagination clicks.

LARAVEL-SIDE FILTERING

```
use Illuminate\Http\Request;

Route::get('/users', function (Request $request) {
    $users = User::query()
        ->when($request->input('search'), function ($query, $search) {
            $query->where('name', 'like', "%{$search}%");
        })
        ->paginate(10)
        ->withQueryString() // keeps ?search=ben on pagination links
        ->through(fn($u) => [
            'id' => $u->id,
            'name' => $u->name,
        ]);

    return Inertia::render('Users/Index', [
        'users' => $users,
        'filters' => $request->only('search'), // echo back to client
    ]);
});
```

SVELTE-SIDE SEARCH INPUT

```
<script>
    import { router } from '@inertiajs/svelte';

    let { users, filters } = $props();
    let search = $state(filters.search ?? '');

    // Re-run on every search change. router.get is debounced below.
    $effect(() =>
        router.get(
            '/users',
            { search },

            preserveState: true, // keep input focus + value
            preserveScroll: true,
            replace: true, // dont pollute browser history
        ),
    );
</script>

<input
    type="text"
    bind:value={search}
    placeholder="Search users..."
```

```
class="border rounded px-3 py-2"
/>
```

```
<!-- table + pagination here, reading users.data and users.links -->
```

PRESERVESTATE IS NON-NEGOTIABLE — Without `preserveState: true`, every keystroke re-renders the page from fresh server props — the search input resets to whatever Laravel sent. You'd type "b", get filtered results, then the input would clear. `preserveState` keeps your local state alive across the request.

FILTERS ROUND-TRIPPING

Notice the controller passes `filters` back to the client, and the page initializes `search` from `filters.search`. This makes the URL-as-state pattern work: refresh the page on `/users?search=ben` and the input shows "ben" again.

18. INERTIA FORMS 101

SETTING UP THE ROUTES

```
use App\Http\Controllers\UserController;

Route::get('/users/create', [UserController::class, 'create']);
Route::post('/users', [UserController::class, 'store']);
```

THE CONTROLLER

```
<?php
namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Inertia\Inertia;

class UserController extends Controller
{
    public function create()
    {
        return Inertia::render('Users/Create');
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'email', 'unique:users'],
            'password' => ['required', 'min:8'],
        ]);

        User::create($validated);

        return redirect('/users')
            ->with('success', 'User created!');
    }
}
```

ALWAYS RETURN A REDIRECT — On success, redirect — never return JSON or `Inertia::render` directly. Inertia automatically follows the redirect, fetches the new page, and you get a clean URL and back-button behavior. Returning JSON breaks the SPA flow.

HASHING PASSWORDS

In modern Laravel, password hashing is handled by an Eloquent cast on the User model. Check `app/Models/User.php`:

```
protected function casts(): array
{
    return [
        'email_verified_at' => 'datetime',
        'password' => 'hashed', // auto-hash on assignment
    ];
}
```

So `User::create(['password' => 'secret'])` stores the bcrypt hash automatically. No need for the old `setPasswordAttribute` mutator from the course.

THE SVELTE FORM (MANUAL APPROACH FIRST)

```
<script>
  import { router } from '@inertiajs/svelte';

  let form = $state({
    name:    '',
    email:   '',
    password: '',
  });

  function submit(e)
    e.preventDefault();
    router.post('/users', form);

</script>

<form onsubmit={submit}>
  <input type="text"      bind:value={form.name}      placeholder="Name" />
  <input type="email"     bind:value={form.email}     placeholder="Email" />
  <input type="password"  bind:value={form.password}  placeholder="Password" />

  <button type="submit">Create User</button>
</form>
```

SVELTE 5 EVENT SYNTAX — Use `onsubmit` (one word) — not `on:submit` (the older Svelte syntax). Same for `onclick`, `oninput`, `onchange`. Svelte 5 unified DOM event handlers with native HTML.

19. DISPLAYING FAILED VALIDATION ERRORS

HOW ERRORS GET TO THE CLIENT

When `$request->validate()` fails, Laravel:

- Flashes the errors to the session
- Redirects back to the previous URL
- Inertia auto-shares `errors` as a prop on every page (via the base middleware)

READING ERRORS IN SVELTE

Two equivalent ways. Via the page store:

```
<script>
  import { page } from '@inertiajs/svelte';
  let errors = $derived($page.props.errors);
</script>

<input bind:value={form.name} />
{#if errors.name}
  <p class="text-red-600 text-sm">{errors.name}</p>
{/if}
```

Or as a normal prop:

```
<script>
  let { errors = {} } = $props();
</script>

<input bind:value={form.email} />
{#if errors.email}
  <p class="text-red-600 text-sm">{errors.email}</p>
{/if}
```

HTML5 REQUIRED IS NOT VALIDATION — Adding required to an input is a nice UX hint, but client-side validation is bypassable in two seconds. Server-side validation is the only validation that matters. Treat HTML5 as decoration.

20. INERTIA'S USEFORM HELPER

THE SPAM-CLICK SUBMIT PROBLEM

With the manual approach, nothing stops a user clicking submit 5 times before the server responds. You'd create 5 users. You need a `processing` flag. Plus tracking errors, resetting after success, showing a "Saved!" message — all manual.

USEFORM SOLVES ALL OF IT

```
<script>
  import { useForm } from '@inertiajs/svelte';

  // useForm returns a Svelte store, so use $form to read/write
  const form = useForm(
    name:      '',
    email:     '',
    password:  '',
  );

  function submit(e)
  {
    e.preventDefault();
    $form.post('/users',
      onSuccess: () => $form.reset(),
    );
  }
</script>

<form onsubmit={submit}>
  <input bind:value={$form.name} />
  {#if $form.errors.name}<p class="text-red-600">{$form.errors.name}</p>{/if}

  <input bind:value={$form.email} />
  {#if $form.errors.email}<p class="text-red-600">{$form.errors.email}</p>{/if}

  <input type="password" bind:value={$form.password} />
  {#if $form.errors.password}<p class="text-red-600">{$form.errors.password}</p>{/if}

  <button type="submit" disabled={$form.processing}>
    {$form.processing ? 'Creating...' : 'Create User'}
  </button>

  {#if $form.recentlySuccessful}
    <span class="text-green-600">Saved!</span>
  {/if}
</form>
```

WHAT YOU GET FOR FREE

- `$form.processing` — true during request, false otherwise
- `$form.errors` — object keyed by field name
- `$form.recentlySuccessful` — true for 2 seconds after success
- `$form.progress` — upload progress object for file uploads
- `$form.isDirty` — true if any field has changed from initial state

- `$form.reset()` — restore initial values
- `$form.post(url) / .put(url) / .patch(url) / .delete(url) / .get(url)`
- `$form.transform(data => ...)` — modify data before sending

WHY THE \$ PREFIX — `useForm` returns a Svelte store. Svelte's store contract requires the `$` prefix to access the current value reactively. The store auto-subscribes and updates when values change.

21. THROTTLE AND DEBOUNCE

THE PROBLEM WITH THE SEARCH BOX

From section 17 — every keystroke fires a request. Typing "benjamin" sends 8 requests in under a second. The server is hammered, results race, and the UI flickers as old slow requests resolve after newer ones.

THROTTLE VS DEBOUNCE — THE REAL DIFFERENCE

- **Throttle** fires the function *at most* every N ms, evenly spaced. Type 10 chars quickly with a 300ms throttle → fires roughly every 300ms.
- **Debounce** waits until the user *stops* typing for N ms, then fires once. Type 10 chars quickly with a 300ms debounce → fires once, 300ms after the last keystroke.
- For a search box, **debounce is what you want**. You don't care about intermediate results, only the final query.

USING LODASH DEBOUNCE

```
<script>
import { router } from '@inertiajs/svelte';
import { debounce } from 'lodash-es';

let { users, filters } = $props();
let search = $state(filters.search ?? '');

// Build the debounced function once
const sendRequest = debounce((value) =>
  router.get(
    '/users',
    { search: value },
    { preserveState: true, preserveScroll: true, replace: true },
  ),
  300);

$effect(() =>
  sendRequest(search);
);
</script>

<input type="text" bind:value={search} placeholder="Search users..." />
```

LODASH-ES FOR TREE-SHAKING — Use `lodash-es` (ES modules) instead of plain `lodash` so Vite can tree-shake unused functions. The `import { debounce } from 'lodash-es'` line only pulls in `debounce`, not the whole library.

22. AUTHENTICATION WITH INERTIA

PROTECTING ROUTES

```
Route::middleware('auth')->group(function () {
    Route::get('/dashboard', [DashboardController::class, 'index']);
    Route::get('/tweets', [TweetController::class, 'index']);
    Route::post('/tweets', [TweetController::class, 'store']);
});

// Public routes
Route::get('/login', [Auth\LoginController::class, 'create'])->name('login');
Route::post('/login', [Auth\LoginController::class, 'store']);
Route::post('/logout', [Auth\LoginController::class, 'destroy'])->middleware('auth');
```

THE LOGIN ROUTE NAME MATTERS — Laravel's auth middleware redirects unauthenticated users to a route named "login". If you don't name your route "login", the redirect breaks with "Route [login] not defined".

THE LOGINCONTROLLER

```
<?php
namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Validation\ValidationException;
use Inertia\Inertia;

class LoginController extends Controller
{
    public function create()
    {
        return Inertia::render('Auth/Login');
    }

    public function store(Request $request)
    {
        $credentials = $request->validate([
            'email' => ['required', 'email'],
            'password' => ['required'],
        ]);

        if (! Auth::attempt($credentials, $request->boolean('remember'))) {
            throw ValidationException::withMessages([
                'email' => 'These credentials do not match our records.',
            ]);
        }

        $request->session()->regenerate();

        return redirect()->intended('/dashboard');
    }

    public function destroy(Request $request)
    {

```

```

    Auth::logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return redirect('/');
}
}

```

THE LOGIN PAGE (NO LAYOUT)

```

<!-- resources/js/Pages/Auth/Login.svelte -->
<script module>
  export const layout = null; // no nav on the login page
</script>

<script>
  import { useForm } from '@inertiajs/svelte';

  const form = useForm(
    email: '',
    password: '',
    remember: false,
  );

  function submit(e)
  {
    e.preventDefault();
    $form.post('/login',
      onFinish: () => $form.reset('password'),
    );
  }
</script>

<div class="min-h-screen flex items-center justify-center bg-gray-50">
  <form onsubmit={submit} class="bg-white p-8 rounded shadow-md w-96">
    <h1 class="text-2xl font-bold mb-6">Sign in</h1>

    <input type="email" bind:value={$form.email} placeholder="Email" class="w-full mb-2
border rounded px-3 py-2" />
    {#if $form.errors.email}<p class="text-red-600 text-sm">{$form.errors.email}</p>{/if}

    <input type="password" bind:value={$form.password} placeholder="Password" class="w-
full mt-3 mb-2 border rounded px-3 py-2" />
    {#if $form.errors.password}<p class="text-red-600 text-sm">{$form.errors.password}</p>{/if}

    <label class="flex items-center mt-3 text-sm">
      <input type="checkbox" bind:checked={$form.remember} class="mr-2" />
      Remember me
    </label>

    <button
      type="submit"
      disabled={$form.processing}
      class="w-full mt-4 bg-blue-600 text-white py-2 rounded disabled:opacity-50"
    >
      {$form.processing ? 'Signing in...' : 'Sign in'}

```



```
    </button>
  </form>
</div>
```

UPDATING DEFAULT LAYOUT LOGIC

Recall section 12 — we wrote `if (page.default.layout === undefined)`. That strict equality with `undefined` is what makes `export const layout = null` work. A simple falsy check would overwrite `null` with the default `Layout`.

SHARING AUTH DATA SAFELY

```
// HandleInertiaRequests::share()

'auth' => [
  'user' => fn() => Auth::check()
    ? [
      'id'      => Auth::user()->id,
      'name'    => Auth::user()->name,
      'username' => Auth::user()->username,
      'email'   => Auth::user()->email,
    ]
    : null,
],
```

WHY EXPLICIT FIELDS OVER ONLY() — `Auth::user()->only(['id', 'name'])` keeps the original DB column names. Explicit mapping lets you rename fields for the client (e.g., `display_name` from `full_name`) and decouples your client from your schema. Renaming the column doesn't break the UI.

LOGOUT BUTTON

```
<NavLink href="/logout" method="post" as="button">
  Log out
</NavLink>
```

23. AUTHORIZATION TIPS

PASSING AUTHORIZATION FROM THE CONTROLLER

You want to hide a "Create User" button from users who can't create users. Pass that decision down as a prop:

```
Route::get('/users', function () {
    return Inertia::render('Users/Index', [
        'users' => User::paginate(10)->through(/* ... */),
        'can'    => [
            'createUser' => Auth::user()?->can('create', User::class),
        ],
    ]);
})->middleware('auth');

<script>
    import { Link } from '@inertiajs/svelte';
    let { users, can } = $props();
</script>

{#if can.createUser}
    <Link href="/users/create" class="btn-primary">Create User</Link>
{/if}
```

UI ALONE IS NOT SECURITY

Hiding the button hides the option, but a malicious user can still POST to `/users` directly. **You must also protect the endpoint.** The UI check is convenience, not protection.

CREATING A POLICY

```
php artisan make:policy UserPolicy --model=User

<?php
namespace App\Policies;

use App\Models\User;

class UserPolicy
{
    public function create(User $user): bool
    {
        return $user->is_admin; // your business logic
    }

    public function edit(User $user, User $target): bool
    {
        return $user->id === $target->id || $user->is_admin;
    }

    public function delete(User $user, User $target): bool
    {
        return $user->is_admin && $user->id !== $target->id;
    }
}
```

TWO WAYS TO PROTECT ROUTES

Middleware syntax:

```
Route::get('/users/create', [UserController::class, 'create'])
    ->middleware('can:create,App\Models\User');
```

Fluent syntax (Jeffrey prefers this):

```
Route::get('/users/create', [UserController::class, 'create'])
    ->can('create', \App\Models\User::class);
```

PER-RECORD AUTHORIZATION

Show an "Edit" button per row, but only if the current user can edit that specific user.

```
'users' => User::paginate(10)->through(fn($u) => [
    'id' => $u->id,
    'name' => $u->name,
    'can' => [
        'edit' => Auth::user()->can('edit', $u),
        'delete' => Auth::user()->can('delete', $u),
    ],
]),
{#each users.data as user}
    <tr>
        <td>{user.name}</td>
        <td>
            {#if user.can.edit}
                <Link href="/users/{user.id}/edit">Edit</Link>
            {/if}
            {#if user.can.delete}
                <Link href="/users/{user.id}" method="delete" as="button">
                    Delete
                </Link>
            {/if}
        </td>
    </tr>
{/each}
```

THE JEFFREY HOMEWORK

At the end of the course, Jeffrey leaves the edit-user flow as an exercise. The pieces:

- Route: `Route::get('/users/{user}/edit', [UserController::class, 'edit'])->can('edit', 'user');`
- Controller: `edit(User $user)` returns `Inertia::render('Users/Edit', ['user' => $user])`
- Policy: `edit(User $auth, User $target)` returns the business rule
- Page: `Users/Edit.svelte` with `useForm($user)` and `$form.put('/users/' + user.id)`

BIG PICTURE — HOW IT ALL CONNECTS

Here's the full request lifecycle of a typical Inertia + Svelte app, end-to-end:

1. THE FIRST REQUEST

- User types `twitter-clone.test/users` into the browser
- Laravel matches the route, runs middleware (auth, CSRF, `HandleInertiaRequests`), hits the controller
- Controller queries DB → paginates → `->through()` to whitelist columns → returns `Inertia::render('Users/Index', [...])`
- No `X-Inertia` header (first visit), so Inertia returns the full HTML shell containing the Svelte app and an embedded `data-page` attribute with the JSON payload
- Browser loads, Vite bundle boots, `createInertiaApp()` reads the `data-page`, mounts `Users/Index.svelte` wrapped in the default Layout, with the page props injected

2. THE NEXT CLICK

- User clicks `<Link href="/users?search=ben">`
- Inertia intercepts → AJAX request with `X-Inertia: true` header → same Laravel controller
- Controller queries DB → returns `Inertia::render()` as before
- Inertia middleware sees `X-Inertia` → returns JSON instead of HTML
- Client receives JSON, swaps the page component's props, persists the Layout (because layout is the parent), updates the URL via `history.pushState`

3. A FORM SUBMISSION

- `useForm` Svelte store holds field state and error state
- User clicks submit → `$form.post('/users')` → AJAX POST with CSRF cookie (automatic) → controller
- Validation fails → Laravel redirects back with errors flashed to session → Inertia auto-shares errors → `$form.errors.email` becomes truthy → red text appears under the input
- Validation succeeds → controller creates record → redirects to `/users` → Inertia follows redirect → user lands on the list page, sees the new record

4. WHERE THE LAYERS SIT

- **Laravel** — routes, controllers, models, validation, policies, sessions, CSRF, business logic
- **Inertia** — the courier service: passes data from controllers to pages, intercepts clicks, manages history
- **Svelte** — the renderer: components, reactivity, runes, DOM updates
- **Vite** — the build tool: hot reload in dev, bundling for prod, no Mix needed
- **Tailwind** — the styling: utility classes everywhere

5. THE CARDINAL RULES

- Never share full Eloquent models — always whitelist columns
- Always `->through()` over a paginator, never `->map()`
- Always return a redirect from non-GET controller actions, never JSON

- Always use `preserveState: true` when re-fetching the same page with new params
- Always validate server-side, never trust client-side validation alone
- Always protect the endpoint with a policy or middleware, never rely on UI hiding alone

6. WHERE TO GO NEXT

- Build the Twitter clone's data model: User, Tweet, Like, Follow, Reply
- Wire up the feed page with pagination + filters
- Add the tweet composer with `useForm`
- Add like buttons using `preserveScroll` POST links
- Add Filament admin for moderating tweets and the auto-ban-words scheduled job
- Add the password reset emails using Laravel's built-in scaffolding

End of notes. Build something good.